# Selective Repeat ARQ (Part 1)

Amitis Mirabedini
Student ID: 402102562

June 25, 2026

# Contents

**1 Introduction** **4**
1.1 Network Parameters . . . 4
1.2 Analytical Calculations . . . 4
1.2.1 Transmission Time ($T_x$) . . . 4
1.2.2 Round-Trip Time (RTT) . . . 4
1.2.3 Bandwidth-Delay Product (BDP) . . . 4
1.2.4 Optimal Window Size ($W_{opt}$) . . . 4
1.3 Channel Error Modeling . . . 5

**2 Implementation** **6**
2.1 Message Definitions . . . 6
2.2 Sender Implementation . . . 7
2.2.1 Individual Frame Timers . . . 7
2.2.2 Timer Lifecycle . . . 8
2.2.3 NAK Interpretation . . . 8
2.3 Receiver Implementation . . . 9
2.3.1 Buffer Arrays . . . 9
2.3.2 Frame Storage . . . 10
2.3.3 NAK Guard . . . 11
2.3.4 In-Order Delivery . . . 11
2.4 Sequence Number Space . . . 12
2.4.1 Sequence Space Examples . . . 12
2.4.2 Why $M \geq 2W$ . . . 13
2.5 Implementation Summary . . . 13

**3 Screenshots - 2nd Part of Deliverables** **14**
3.1 Network Topology . . . 14
3.2 Simulation Runtime Clock Configuration . . . 15
3.3 Module Parameters and PER Configuration . . . 15
3.3.1 Error-Free Configuration (PER = 0.0) . . . 16
3.3.2 Lossy Configuration (PER = 0.2 on Forward Channel) . . . 16
3.3.3 Lossy Configuration (PER = 0.3 on Forward Channel) . . . 17
3.4 Configuration Summary . . . 17

**4 Sample Logs** **18**
4.1 Error-Free Scenario (PER = 0.0, W = 4) . . . 18
4.1.1 Initial Transmission (Frames 0-3) . . . 18
4.1.2 Receiver Processing (Frames 0-3) and ACKs . . . 18
4.1.3 Continuing Transmission (Frames 4-11) . . . 19
4.1.4 Complete Error-Free Sequence . . . 20
4.1.5 Sample Error-Free Log . . . 21
4.2 Lossy Scenario (PER = 0.2, W = 20) . . . 21
4.2.1 Out-of-Order Buffering and NAK Handling . . . 21
4.2.2 Timeout Handling . . . 22
4.2.3 Batch Delivery After NAK Recovery . . . 23
4.2.4 Complete Lossy Scenario Overview . . . 24

4.3 Required Log Format Summary . . . . . 26

**5 Timeout Analysis** **27**
5.1 Timeout Calculation . . . . . 27
5.2 Timeout Determination . . . . . 27
5.3 Effect of Timeout Below RTT (e.g., 90 ms) . . . . . 28
5.3.1 Premature Timeouts . . . . . 28
5.3.2 Consequences . . . . . 28
5.3.3 Log Example (Hypothetical) . . . . . 28
5.4 Effect of Timeout Above RTT (e.g., 200 ms) . . . . . 28
5.4.1 Long Idle Gaps . . . . . 29
5.4.2 Consequences . . . . . 29
5.4.3 Log Example (Hypothetical) . . . . . 29
5.5 Summary: Timeout Trade-offs . . . . . 30
5.6 Why 105 ms is Optimal for This Simulation . . . . . 30
5.7 Effect of Timeout on Measured Utilization . . . . . 30

**6 Utilization Analysis** **31**
6.1 Theoretical Utilization . . . . . 31
6.1.1 Parameter Calculations . . . . . 31
6.1.2 Error-Free Channel ($p = 0.0$) . . . . . 31
6.1.3 Lossy Channel ($p = 0.3$) . . . . . 32
6.2 Measured Utilization . . . . . 32
6.2.1 Error-Free Channel ($p = 0.0$) . . . . . 33
6.2.2 Lossy Channel ($p = 0.3$) . . . . . 33
6.3 Comparison: Custom Signal vs. Built-in Statistics . . . . . 33
6.4 Scalar and Vector Statistics . . . . . 34
6.4.1 Scalar Statistics . . . . . 34
6.4.2 Vector Statistics . . . . . 34
6.5 Analysis of Discrepancies . . . . . 35
6.5.1 Discrepancy for W = 4, $p = 0.3$ . . . . . 36
6.5.2 Discrepancy for W = 20, $p = 0.3$ . . . . . 36
6.5.3 Discrepancy for W = 120, $p = 0.3$ . . . . . 37
6.6 Summary of Discrepancy Causes . . . . . 37
6.7 Key Observations . . . . . 38

**7 Discussion: Physical Limitations of High-Latency Links** **39**
7.1 The Problem: $T_p \gg T_x$ . . . . . 39
7.2 Window Size $W = 4$: Poor Utilization Despite Error-Free Channel . . . . 39
7.3 Window Size $W = 120$: Saturating the Link . . . . . 40
7.4 Comparison of Idle States . . . . . 40
7.5 Why $W = 4$ Yields Poor Results . . . . . 40
7.6 How Expanding to $W = 120$ Alters Idle States . . . . . 41
7.7 Impact on Lossy Scenarios . . . . . 41
7.8 Summary . . . . . 41

# 1 Introduction

This report presents the implementation and performance analysis of Selective Repeat ARQ (Tanenbaum's Protocol 6) in OMNeT++. The simulation models a high-latency, high-bandwidth network resembling a satellite or transcontinental link, where the propagation delay is significantly longer than the frame transmission time.

## 1.1 Network Parameters

The physical channel is configured with the following baseline parameters:

| Parameter | Value |
|---|---|
| Channel Bit Rate | 100 kbps |
| Frame Payload Length | 100 bits |
| One-Way Propagation Delay ($T_p$) | 50 ms |
| Simulation Time | 100 s |

Table 1: Network Parameters

## 1.2 Analytical Calculations

The following values were calculated using the formulas provided in the assignment:

### 1.2.1 Transmission Time ($T_x$)

The transmission time is the time required to push a single 100-bit frame onto the wire:

$$T_x = \frac{\text{Frame Length}}{\text{Bit Rate}} = \frac{100 \text{ bits}}{100 \times 10^3 \text{ bps}} = 1 \text{ ms}$$

### 1.2.2 Round-Trip Time (RTT)

The Round-Trip Time is the total elapsed time between sending a frame and receiving its acknowledgment under ideal conditions:

$$RTT = 2 \times T_p + T_x = 2(50 \text{ ms}) + 1 \text{ ms} = 101 \text{ ms}$$

### 1.2.3 Bandwidth-Delay Product (BDP)

The Bandwidth-Delay Product represents the total capacity of the link in bits:

$$BDP = \text{Bit Rate} \times RTT = 100 \times 10^3 \text{ bps} \times 0.101 \text{ s} = 10\,100 \text{ bits}$$

### 1.2.4 Optimal Window Size ($W_{opt}$)

The optimal window size is the number of unacknowledged frames required to keep the link 100% saturated:

$$W_{opt} = \frac{BDP}{\text{Frame Length}} = \frac{10100}{100} = 101$$

Thus, a window size of **101** frames is required to keep the link fully saturated.

## 1.3 Channel Error Modeling

The protocol is evaluated under two distinct channel behaviors:

- **Error-Free Channel**: A baseline environment with 0% packet loss to analyze maximum theoretical capacity and window saturation.
- **Lossy Channel**: A noisy transmission medium with Packet Error Rate (PER) of 0.2 and 0.3, applied only on the DATAFRAME path to simulate stochastic packet loss.

# 2 Implementation

This section describes the implementation of the Selective Repeat protocol, including the message definitions, individual frame timer management, receiver buffering, and sequence number space expansion.

## 2.1 Message Definitions

A unified message structure was defined to handle all three frame types (DATA, ACK, and NAK) using a single packet definition. The `type` field acts as a discriminator, allowing both the sender and receiver to process all frame types with a common message class.

```
enum SRFrameType {
    DATA_FRAME = 0;
    ACK_FRAME = 1;
    NAK_FRAME = 2;
}

packet SRMessage {
    int type;         // DATA_FRAME, ACK_FRAME, or NAK_FRAME
    int seqNum;       // Sequence number (used by DATA frames)
    int ackNum;       // Cumulative ACK or specific NAK sequence
   boundary
    string payload;  // 100-bit data string
}
```

Listing 1: SRMessage.msg

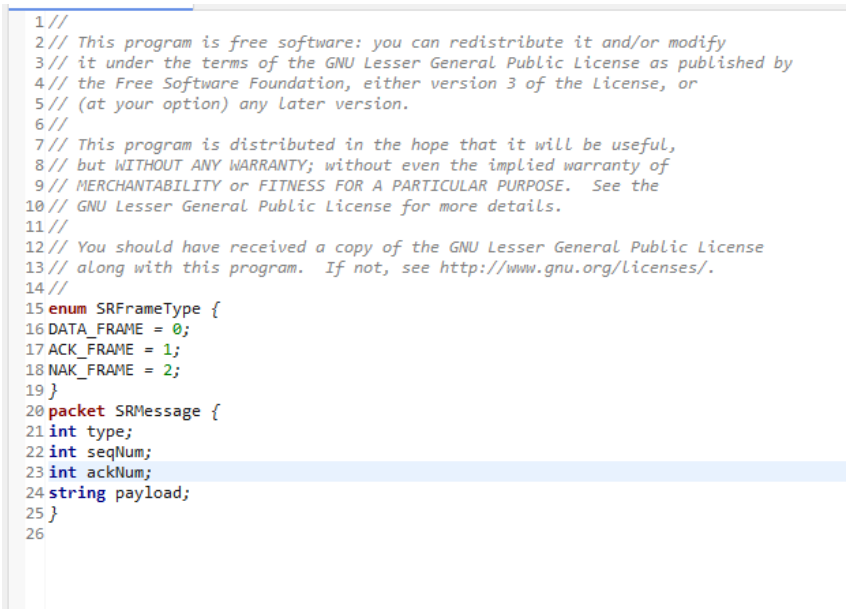


Figure 1: Screenshot of SRMessage.msg file

The fields are used as follows:

| Frame Type | type | seqNum | ackNum | payload |
|---|---|---|---|---|
| DATA | DATA_FRAME | Used | Unused | Used |
| ACK | ACK_FRAME | Unused | Used | Unused |
| NAK | NAK_FRAME | Unused | Used | Unused |

Table 2: Field Usage by Frame Type

The unified message structure simplifies the implementation by allowing both `SrcNode` and `DstNode` to use a single `check_and_cast<SRMessage*>()` operation in `handleMessage()`, rather than requiring separate message classes for each frame type.

## 2.2 Sender Implementation

The sender module maintains an array of timers and buffers for outstanding frames. This section describes how individual frame timers are managed using OMNeT++'s `cMessage` contexts.

### 2.2.1 Individual Frame Timers

Instead of using a single timeout timer for the entire window (as in Go-Back-N), Selective Repeat requires independent timers for each outstanding frame. This is implemented using an array of `cMessage` timers mapped to sequence number slots using OMNeT++'s `setContextPointer()` mechanism.

```
int seqSpaceSize = maxSeqNum + 1;
tx_timers.resize(seqSpaceSize, nullptr);
for (int i = 0; i < seqSpaceSize; i++) {
    tx_timers[i] = new cMessage("timeout");
    tx_timers[i]->setContextPointer((void*)(uintptr_t)i);
}
```

Listing 2: Timer Initialization in SrcNode.cc

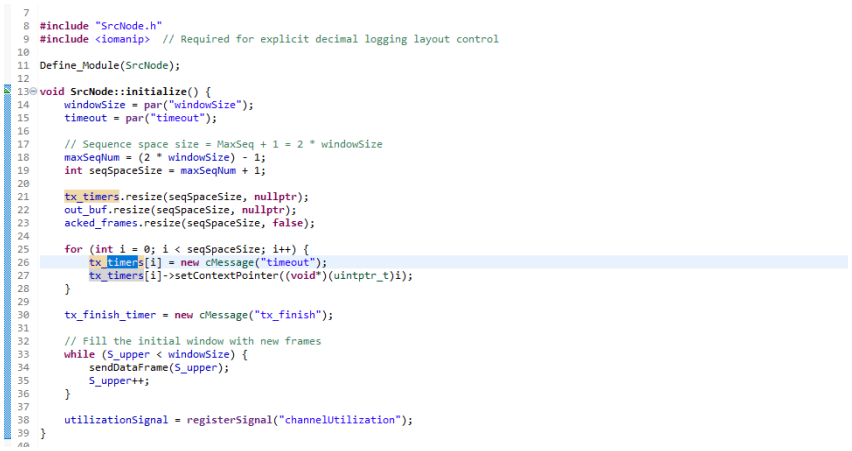


Figure 2: Screenshot of timer initialization code

When a frame is sent, the timer for that specific sequence number slot is started. When a timer fires, the sequence number is retrieved via `getContextPointer()`:

```
int slot = (int)(uintptr_t)msg->getContextPointer();
if (out_buf[slot] != nullptr && !acked_frames[slot]) {
    int seqToResend = out_buf[slot]->getSeqNum();
    EV << "src: timeout on frame " << seqToResend
       << " - RETRANSMITTING ONLY THIS FRAME\n";
    mac_queue.push(std::make_pair(out_buf[slot]->dup(), false));
    startTransmission();
}
```

Listing 3: Timeout Handler


Figure 3: Screenshot of timeout handler code

### 2.2.2 Timer Lifecycle

The lifecycle of each timer is as follows:

| Event | Action |
|---|---|
| Frame sent | Timer started: `scheduleAt(simTime() + timeout, tx_timers[slot])` |
| ACK received | Timer cancelled: `cancelEvent(tx_timers[slot])` |
| Timeout fires | Retrieve slot via `getContextPointer()`, retransmit frame |
| NAK received | Immediately retransmit, cancel and restart timer |
| Window slides | All timers for acknowledged frames are cancelled |

Table 3: Timer Lifecycle Events

### 2.2.3 NAK Interpretation

When a NAK is received, the sender immediately retransmits the requested frame without waiting for its timer to expire:

```
else if (incoming->getType() == NAK_FRAME) {
    int nakNum = incoming->getAckNum();
    EV << "src: received NAK for frame " << nakNum << ". Retransmitting
   .\n";

    if (nakNum >= S_lower && nakNum < S_upper) {
        int slot = nakNum % (maxSeqNum + 1);
        if (out_buf[slot] != nullptr) {
            mac_queue.push(std::make_pair(out_buf[slot]->dup(), false))
   ;
```

```
                startTransmission();
            }
        }
}
```

Listing 4: NAK Handling in SrcNode.cc

```
        if (incoming->getType() == ACK_FRAME) {
            int ackNum = incoming->getAckNum();
            EV << "src: received ACK. Frame " << ackNum << " confirmed.\n";

            if (ackNum >= S_lower && ackNum < S_upper) {
                while (S_lower <= ackNum) {
                    int slot = S_lower % (maxSeqNum + 1);

                    totalGoodputBits += 100;

                    if (out_buf[slot] != nullptr) {
                        delete out_buf[slot];
                        out_buf[slot] = nullptr;
                    }
                    acked_frames[slot] = false;
                    if (tx_timers[slot]->isScheduled()) {
                        cancelEvent(tx_timers[slot]);
                    }
                    S_lower++;
                }
                while (S_upper < S_lower + windowSize) {
                    sendDataFrame(S_upper);
                    S_upper++;
                }
            }
        }
        else if (incoming->getType() == NAK_FRAME) {
            int nakNum = incoming->getAckNum();
            EV << "src: received NAK for frame " << nakNum << ". Retransmitting.\n";

            if (nakNum >= S_lower && nakNum < S_upper) {
                int slot = nakNum % (maxSeqNum + 1);
                if (out_buf[slot] != nullptr) {
                    mac_queue.push(std::make_pair(out_buf[slot]->dup(), false));
                    startTransmission();
                }
            }
        }
        delete incoming;
    }
}
```

Figure 4: Screenshot of NAK handling code

## 2.3 Receiver Implementation

The receiver buffers out-of-order frames and uses a NAK guard to prevent NAK storms. This section describes how the receiver's out-of-order buffer array tracks gaps.

### 2.3.1 Buffer Arrays

Two parallel arrays track buffered frames:

```
std::vector<bool> arrived;          // Tracks which frames have arrived
std::vector<SRMessage*> in_buf;     // Stores the actual frame data
```

Listing 5: Receiver Buffer Declaration in DstNode.h

```
#ifndef DSTNODE_H_
#define DSTNODE_H_

#include <omnetpp.h>
#include <vector>
#include <queue>
#include "SRMessage_m.h"

using namespace omnetpp;

class DstNode : public cSimpleModule
{
private:
    int windowSize;
    int frame_expected = 0;
    bool no_nak = true;
    int too_far = 0;

    std::vector<bool> arrived;
    std::vector<SRMessage*> in_buf;

    // Transmission queue for ACK/NAK frames
    std::queue<SRMessage*> txQueue;
    cMessage *tx_finish_timer = nullptr;

    void sendFeedback(int type, int num);
    void startTransmission();

protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
    virtual void finish() override;
};

#endif /* DSTNODE_H_ */
```

Figure 5: Screenshot of buffer declarations in DstNode.h

The `arrived` vector uses boolean flags to indicate whether a frame has been received, while `in_buf` stores the actual frame data for later in-order delivery.

### 2.3.2 Frame Storage

When a DATA frame arrives within the current window, it is stored in the buffer:

```
if (seq >= frame_expected && seq < too_far) {
    int slot = seq % seqSpaceSize;
    if (!arrived[slot]) {
        arrived[slot] = true;
        in_buf[slot] = incoming->dup();
        EV << "dst: buffering frame " << seq << "\n";
    }
}
```

Listing 6: Buffer Storage in DstNode.cc

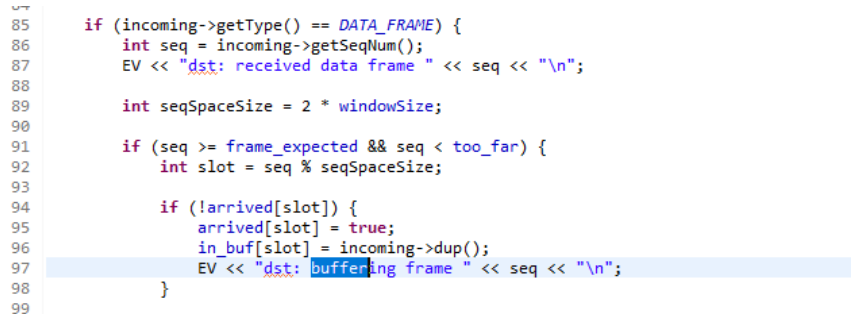


Figure 6: Screenshot of buffer storage logic

The modulo operation slot = seq%seqSpaceSize maps each sequence number to its buffer slot, ensuring that frames wrap around correctly within the sequence space.

### 2.3.3 NAK Guard

The `no_nak` flag ensures only one NAK is sent per missing frame, preventing a NAK storm:

```
if (seq > frame_expected && no_nak) {
    EV << "dst: frame out of order. Sending NAK for " << frame_expected
    << "\n";
    sendFeedback(NAK_FRAME, frame_expected);
    no_nak = false;  // Prevent duplicate NAKs
}
```

Listing 7: NAK Guard Logic

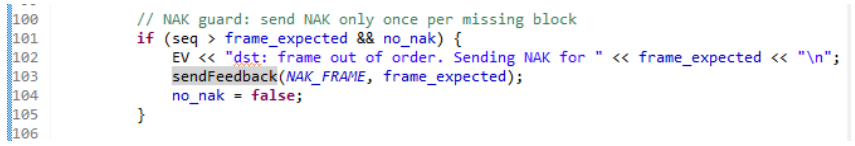


Figure 7: Screenshot of NAK guard logic

Without this guard, the receiver would send a NAK for the same missing frame every time a new out-of-order frame arrives, flooding the reverse channel with redundant NAKs.

### 2.3.4 In-Order Delivery

When the missing frame arrives, all consecutive buffered frames are delivered to the network layer in order:

```
while (arrived[frame_expected % seqSpaceSize]) {
    int currSlot = frame_expected % seqSpaceSize;
    EV << "dst: passing frame " << frame_expected << " to network layer
    \n";
    delete in_buf[currSlot];
    in_buf[currSlot] = nullptr;
    arrived[currSlot] = false;
    frame_expected++;
    too_far++;
    no_nak = true;  // Reset NAK guard
}
```

Listing 8: In-Order Delivery

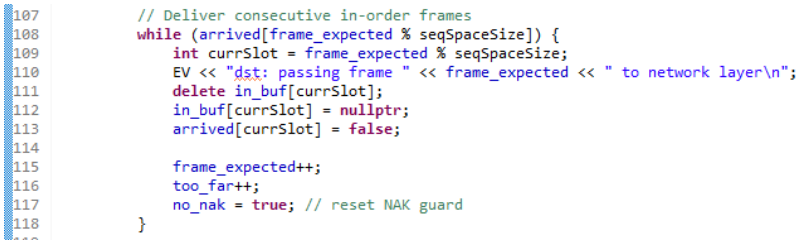


Figure 8: Screenshot of in-order delivery logic

When the window slides, both `frame_expected` and `too_far` are incremented, and the `no_nak` flag is reset to allow future NAKs if new gaps are detected.

## 2.4 Sequence Number Space

To prevent window overlap ambiguity, the sequence number space must be at least twice the window size:

$$\text{MAX\_SEQ} = 2W - 1$$

```
maxSeqNum = (2 * windowSize) - 1;
int seqSpaceSize = maxSeqNum + 1;  // Sequence space size = 2W
```

Listing 9: Sequence Space Initialization

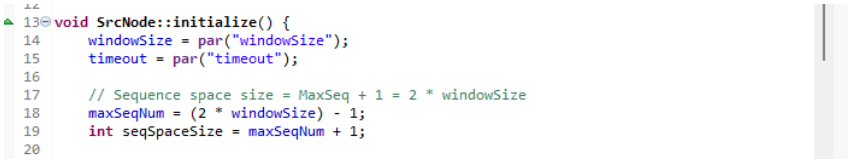


Figure 9: Screenshot of sequence space initialization

### 2.4.1 Sequence Space Examples

The following table shows the sequence number space for each window size used in the simulation:

| Window Size (W) | maxSeqNum | Sequence Space Size | Bits Required |
|---|---|---|---|
| 4 | 7 | 8 (0–7) | 3 bits |
| 20 | 39 | 40 (0–39) | 6 bits |
| 120 | 239 | 240 (0–239) | 8 bits |

Table 4: Sequence Space Configurations

### 2.4.2 Why $M \geq 2W$

The condition $M \geq 2W$ is required to prevent window overlap ambiguity. Consider a scenario where:

1. The receiver has acknowledged frames 0–3 and is waiting for frames 4–7.
2. All ACKs are lost, and the sender retransmits frame 0.
3. Without a large enough sequence space, the receiver might mistake the retransmitted frame 0 for a new frame 8.

By ensuring $M \geq 2W$, even in the worst case where the sender and receiver windows are completely misaligned, there is no ambiguity between new frames and retransmitted old frames. This satisfies the Selective Repeat requirement that the window size must be less than or equal to half of the sequence number space:

$$W \leq \frac{\text{MAX\_SEQ} + 1}{2}$$

The sender and receiver both use modulo indexing to map sequence numbers to buffer slots:

```
// Sender indexing
int slot = seqNum % (maxSeqNum + 1);

// Receiver indexing
int slot = seq % seqSpaceSize;
```

Listing 10: Modulo Indexing

## 2.5 Implementation Summary

| Component | Implementation | Key Data Structure |
|---|---|---|
| Message Definition | Unified `SRMessage` packet | .msg file |
| Timer Management | Individual timers with `setContextPointer()` | `std::vector<cMessage*>` |
| Sender Buffer | Stores copies of outstanding frames | `std::vector<SRMessage*>` |
| Receiver Buffer | Tracks gaps and stores out-of-order frames | `std::vector<bool>` + `std::v` |
| NAK Guard | Prevents NAK storms | `bool no_nak` |
| Sequence Space | $M = 2W$ to prevent ambiguity | `maxSeqNum = 2W - 1` |

Table 5: Implementation Components Summary

# 3 Screenshots - 2nd Part of Deliverables

This section presents screenshots of the simulation environment, including the network topology, runtime clock configuration, and module parameters with PER configuration.

## 3.1 Network Topology

The network topology consists of two nodes, `SrcNode` and `DstNode`, connected by a bidirectional `TranscontinentalLink` channel. The channel is configured with a data rate of 100 kbps, a propagation delay of 50 ms, and configurable Packet Error Rate (PER) on the forward channel only.

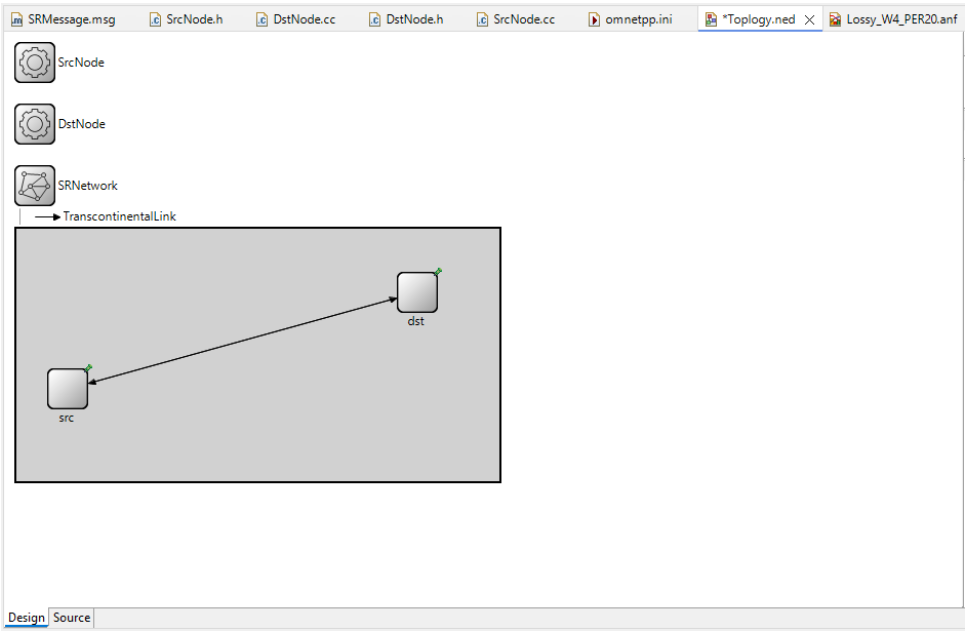


Figure 10: Network Topology Design in OMNeT++

```
//
// You should have received a copy of the GNU Lesser General Public License
// along with this program.  If not, see http://www.gnu.org/licenses/.
//
simple SrcNode
{
    parameters:
        int windowSize;
        double timeout @unit(s);
        @signal[channelUtilization](type="double");
        @statistic[channelUtilization](title="Channel Utilization Efficiency"; source="channelUtilization"; record=vector,last);
    gates:
        input in;
        output out;
}

simple DstNode
{
    parameters:
        int windowSize;
    gates:
        input in;
        output out;
}

network SRNetwork
{
    @display("bg0=471,246");
    types:
        channel TranscontinentalLink extends ned.DatarateChannel
        {
            datarate = 100kbps;
            delay = 50ms;
            per = default(0.0);
        }
    submodules:
        src: SrcNode {
            @display("p=50,156");
        }
        dst: DstNode {
            @display("p=391,62");
        }
    connections:
        src.out --> TranscontinentalLink --> dst.in;
        dst.out --> TranscontinentalLink --> src.in;
}
```

Figure 11: Network Topology Code in OMNeT++

## 3.2 Simulation Runtime Clock Configuration

The simulation is configured with a total duration limit of 100 s, as specified in the assignment. The runtime clock configuration ensures that all simulations terminate after exactly 100 seconds of simulated time.

```
[General]
network = SRNetwork
sim-time-limit = 100s

# Timeout Configuration Analysis:
# Ideal RTT = 2 * Tp + Tx = (2 * 50ms) + 1ms = 101ms.
# To account for software handling, the timer is set to 105ms (0.105s).
**.src.timeout = 0.105s

```

Figure 12: Simulation runtime clock configuration (sim-time-limit = 100s)

## 3.3 Module Parameters and PER Configuration

The module parameters for both `SrcNode` and `DstNode` are configured in the `omnetpp.ini` file. The PER is applied **only on the forward channel** (data frames from SrcNode to DstNode). The reverse channel (ACK/NAK frames from DstNode to SrcNode) has PER = 0.0, ensuring that acknowledgments are always delivered successfully.

- **Forward Channel** (`src.out.channel.per`): Data frames from SrcNode to DstNode (PER = 0.0, 0.2, or 0.3)
- **Reverse Channel** (`dst.out.channel.per`): ACK/NAK frames from DstNode to SrcNode (**PER = 0.0 always**)

**Note:** This unidirectional loss model matches the assumptions of the theoretical utilization formula, which considers only data frame losses and assumes perfect acknowledgments.

### 3.3.1 Error-Free Configuration (PER = 0.0)

```
# ============================================================
# SECTION 1: ERROR-FREE SCENARIOS (PER = 0.0)
# ============================================================

[Config ErrorFree_W4]
description = "Error-Free Link, Small Window Size (W=4)"
**.src.windowSize = 4
**.dst.windowSize = 4
**.src.out.channel.per = 0.0
**.dst.out.channel.per = 0.0

[Config ErrorFree_W20]
description = "Error-Free Link, Medium Window Size (W=20)"
**.src.windowSize = 20
**.dst.windowSize = 20
**.src.out.channel.per = 0.0
**.dst.out.channel.per = 0.0

[Config ErrorFree_W120]
description = "Error-Free Link, Optimal Saturation Window Size (W=120)"
**.src.windowSize = 120
**.dst.windowSize = 120
**.src.out.channel.per = 0.0
**.dst.out.channel.per = 0.0

```

Figure 13: Module parameters for error-free configuration (PER = 0.0)

### 3.3.2 Lossy Configuration (PER = 0.2 on Forward Channel)

```
# ============================================================
# SECTION 2: LOSSY SCENARIOS (PER = 0.2)
# ============================================================

[Config Lossy_W4_PER20]
description = "Noisy Stochastic Loss Link, Small Window Size (W=4), PER=0.2"
**.src.windowSize = 4
**.dst.windowSize = 4
**.src.out.channel.per = 0.2
**.dst.out.channel.per = 0.0

[Config Lossy_W20_PER20]
description = "Noisy Stochastic Loss Link, Medium Window Size (W=20), PER=0.2"
**.src.windowSize = 20
**.dst.windowSize = 20
**.src.out.channel.per = 0.2
**.dst.out.channel.per = 0.0

[Config Lossy_W120_PER20]
description = "Noisy Stochastic Loss Link, Large Window Size (W=120), PER=0.2"
**.src.windowSize = 120
**.dst.windowSize = 120
**.src.out.channel.per = 0.2
**.dst.out.channel.per = 0.0

```

Figure 14: Module parameters for lossy configuration (Forward PER = 0.2, Reverse PER = 0.0)

### 3.3.3 Lossy Configuration (PER = 0.3 on Forward Channel)

```
60 # ============================================================
61 # SECTION 3: LOSSY SCENARIOS (PER = 0.3)
62 # ============================================================
63
64 [Config Lossy_W4_PER30]
65 description = "Noisy Stochastic Loss Link, Small Window Size (W=4), PER=0.3"
66 **.src.windowSize = 4
67 **.dst.windowSize = 4
68 **.src.out.channel.per = 0.3
69 **.dst.out.channel.per = 0.0
70
71 [Config Lossy_W20_PER30]
72 description = "Noisy Stochastic Loss Link, Medium Window Size (W=20), PER=0.3"
73 **.src.windowSize = 20
74 **.dst.windowSize = 20
75 **.src.out.channel.per = 0.3
76 **.dst.out.channel.per = 0.0
77
78 [Config Lossy_W120_PER30]
79 description = "Noisy Stochastic Loss Link, Large Window Size (W=120), PER=0.3"
80 **.src.windowSize = 120
81 **.dst.windowSize = 120
82 **.src.out.channel.per = 0.3
83 **.dst.out.channel.per = 0.0
```

Figure 15: Module parameters for lossy configuration (Forward PER = 0.3, Reverse PER = 0.0)

## 3.4 Configuration Summary

The following configurations are evaluated in this simulation. The PER is applied only on the forward channel (data frames).

| Config | Window Size (W) | Forward PER | Reverse PER |
|---|---|---|---|
| ErrorFree_W4 | 4 | 0.0 | 0.0 |
| ErrorFree_W20 | 20 | 0.0 | 0.0 |
| ErrorFree_W120 | 120 | 0.0 | 0.0 |
| Lossy_W4_PER20 | 4 | 0.2 | 0.0 |
| Lossy_W20_PER20 | 20 | 0.2 | 0.0 |
| Lossy_W120_PER20 | 120 | 0.2 | 0.0 |
| Lossy_W4_PER30 | 4 | 0.3 | 0.0 |
| Lossy_W20_PER30 | 20 | 0.3 | 0.0 |
| Lossy_W120_PER30 | 120 | 0.3 | 0.0 |

Table 6: Simulation Configurations (Data Loss Only)

# 4 Sample Logs

This section presents sample logs from the simulation that demonstrate correct protocol operation under both error-free and lossy scenarios. The logs follow the required format and capture successful frame delivery, out-of-order buffering, NAK handling, and timeouts.

## 4.1 Error-Free Scenario (PER = 0.0, W = 4)

The error-free scenario was run with configuration `ErrorFree_W4`, where the Packet Error Rate (PER) is set to 0.0 on both the forward and reverse channels. With no packet loss, all frames are delivered successfully on the first attempt. The window size is set to W = 4.

### 4.1.1 Initial Transmission (Frames 0-3)

Figure 16 shows the initial transmission of frames 0 through 3 from the sender to the receiver.

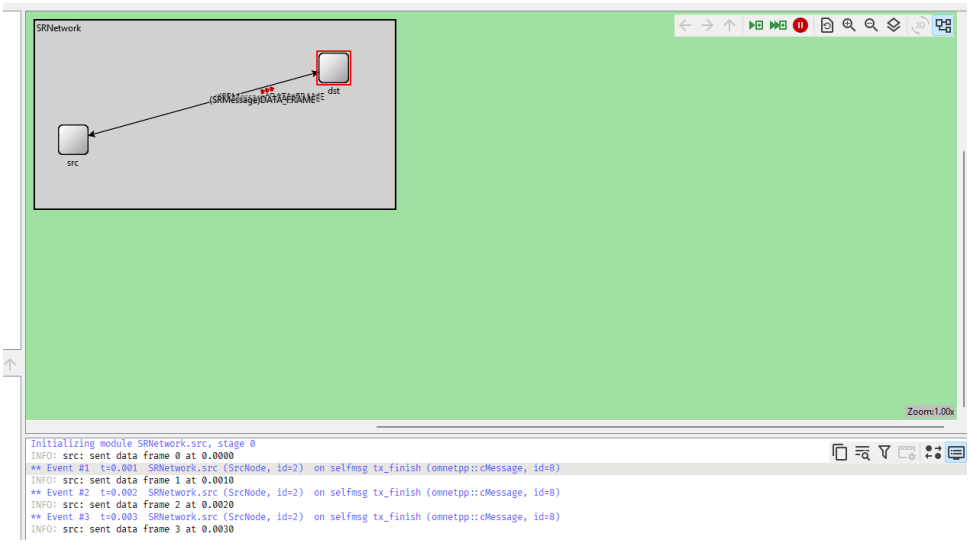


Figure 16: Initial transmission of frames 0-3 in error-free scenario (W=4)

The sender transmits four frames (0, 1, 2, 3) back-to-back, each taking 1 ms to transmit. This fills the window of size W=4. The transmission completes at 0.003 s.

### 4.1.2 Receiver Processing (Frames 0-3) and ACKs

Figure 17 shows the receiver processing frames 0 through 3 in order, and the sender receiving acknowledgments.

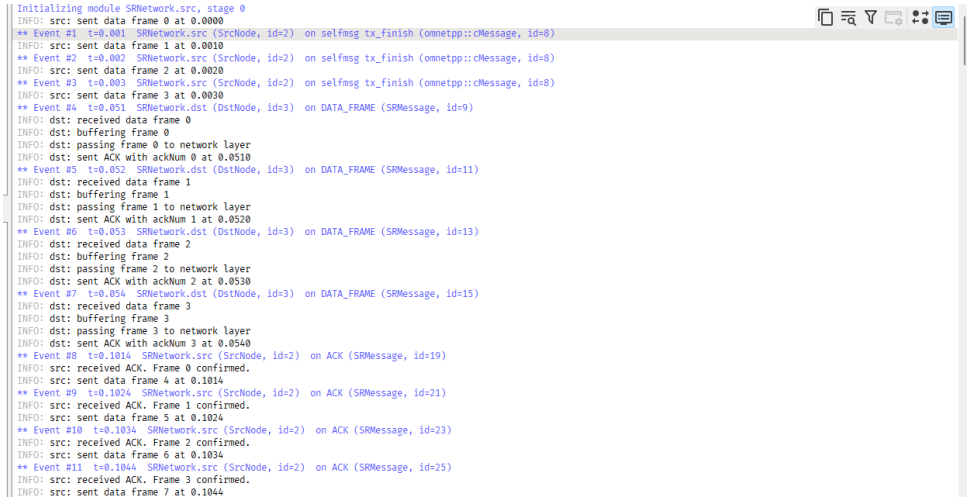


Figure 17: Receiver processing frames 0-3 and ACKs in error-free scenario

After the propagation delay of 50 ms, the receiver starts receiving frames at approximately 51 ms. For each received frame, the receiver:

1. Logs the received frame: `dst: received data frame <seq>`
2. Buffers the frame: `dst: buffering frame <seq>`
3. Delivers the frame to the network layer: `dst: passing frame <seq> to network layer`
4. Sends a cumulative ACK: `dst: sent ACK with ackNum <seq> at <time>`

The sender receives ACKs for frames 0 through 3 at approximately 101.4 ms after each frame was sent, which matches the theoretical RTT calculation:

$$RTT = T_x^{\text{DATA}} + T_p + T_p + T_x^{\text{ACK}} = 1 + 50 + 50 + 0.4 = 101.4\,\text{ms}$$

Upon receiving each ACK, the sender slides its window forward and transmits the next available frame (frames 4, 5, 6, 7).

### 4.1.3 Continuing Transmission (Frames 4-11)

Figure 18 shows the receiver processing frames 4 through 7 and the sender receiving ACKs and transmitting frames 8 through 11.


Figure 18: Continuing transmission: frames 4-7 received, ACKs 4-7, frames 8-11 sent

The receiver processes frames 4-7 at approximately 152 ms to 155 ms, and the sender receives ACKs 4-7 at approximately 202 ms to 205 ms, sending frames 8-11 immediately after each ACK.

### 4.1.4 Complete Error-Free Sequence

Figure 19 shows the complete error-free sequence, including the continuation of frame reception.

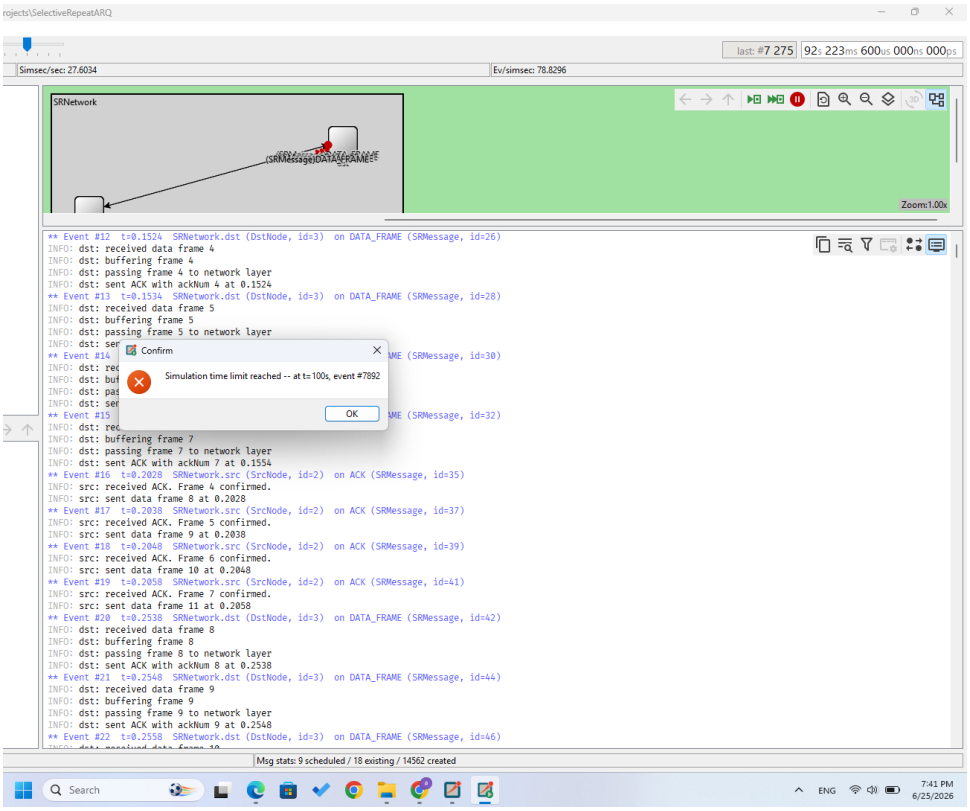


Figure 19: Complete error-free sequence showing frames 4-10

### 4.1.5 Sample Error-Free Log

The following logs demonstrate the complete successful delivery of frame 0 in the error-free scenario:

```
src: sent data frame 0 at 0.0000
dst: received data frame 0
dst: buffering frame 0
dst: passing frame 0 to network layer
dst: sent ACK with ackNum 0 at 0.0510
src: received ACK. Frame 0 confirmed.
```

**Explanation of each log entry:**

- `src: sent data frame 0 at 0.0000` – Sender transmits frame 0 at the start of the simulation.
- `dst: received data frame 0` – Receiver detects the arrival of frame 0 after propagation delay.
- `dst: buffering frame 0` – Receiver stores the frame in its buffer.
- `dst: passing frame 0 to network layer` – Receiver delivers the frame to the upper layer.
- `dst: sent ACK with ackNum 0 at 0.0510` – Receiver sends cumulative ACK for frame 0.
- `src: received ACK. Frame 0 confirmed.` – Sender receives the ACK and marks frame 0 as confirmed.

## 4.2 Lossy Scenario (PER = 0.2, W = 20)

The lossy scenario was run with configuration `Lossy_W20_PER20`, where the Packet Error Rate (PER) is set to 0.3 on the forward channel only. This causes some data frames to be lost, triggering out-of-order buffering, NAK handling, and timeouts.

### 4.2.1 Out-of-Order Buffering and NAK Handling

In this scenario, frame 15 is lost during transmission. When frames 16, 17, 18, and 19 arrive at the receiver, they are out of order. The receiver buffers these frames and sends a NAK for the missing frame 15.

Figure 21 shows the out-of-order buffering and NAK handling events.

```
** Event #35  t=0.066  SRNetwork.dst (DstNode, id=3)  on DATA_FRAME (SRMessage, id=71)
** Event #36  t=0.067  SRNetwork.dst (DstNode, id=3)  on DATA_FRAME (SRMessage, id=73)
INFO: dst: received data frame 16
INFO: dst: buffering frame 16
INFO: dst: frame out of order. Sending NAK for 15
INFO: dst: sent ACK with ackNum 14 at 0.0670
** Event #37  t=0.0674  SRNetwork.dst (DstNode, id=3)  on selfmsg tx_finish (omnetpp::cMessage, id=81)
** Event #38  t=0.068  SRNetwork.dst (DstNode, id=3)  on DATA_FRAME (SRMessage, id=75)
INFO: dst: received data frame 17
INFO: dst: buffering frame 17
INFO: dst: sent ACK with ackNum 14 at 0.0680
** Event #39  t=0.069  SRNetwork.dst (DstNode, id=3)  on DATA_FRAME (SRMessage, id=77)
INFO: dst: received data frame 18
INFO: dst: buffering frame 18
INFO: dst: sent ACK with ackNum 14 at 0.0690
** Event #40  t=0.07  SRNetwork.dst (DstNode, id=3)  on DATA_FRAME (SRMessage, id=79)
INFO: dst: received data frame 19
INFO: dst: buffering frame 19
INFO: dst: sent ACK with ackNum 14 at 0.0700
```

Figure 20: Out-of-order buffering and NAK handling for frame 15

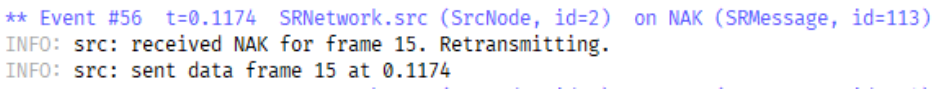


Figure 21: Out-of-order buffering and NAK handling for frame 15

The key log entries are:

```
dst: received data frame 16
dst: buffering frame 16
dst: frame out of order. Sending NAK for 15
dst: sent ACK with ackNum 14 at 0.0670
```

**Explanation:**

- Frame 15 is lost (no log entry for frame 15 at the receiver).
- Frame 16 arrives first out-of-order. The receiver detects the gap because `seq=16 > frame_expected=15`.
- Since `no_nak = true`, the receiver sends a NAK for the missing frame 15.
- The NAK guard (`no_nak = false`) prevents duplicate NAKs when frames 17, 18, and 19 arrive.

When the sender receives the NAK:

```
src: received NAK for frame 15. Retransmitting.
src: sent data frame 15 at 0.1174
```

This demonstrates the sender's immediate retransmission of the requested frame without waiting for a timeout.

### 4.2.2 Timeout Handling

When ACKs for frames 16-19 are lost (or these frames are themselves lost), the sender's individual timers expire, triggering timeouts.

Figure 22 shows the timeout events for frames 16-19.

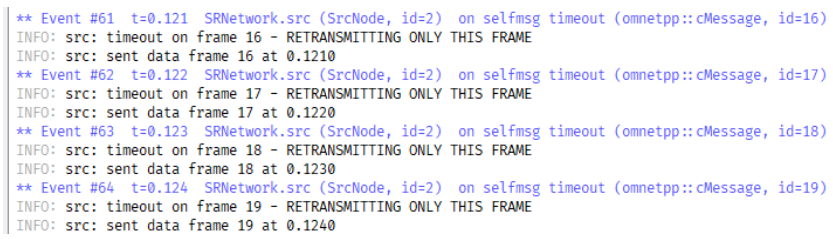


Figure 22: Timeout handling for frames 16-19

The key log entries are:

```
src: timeout on frame 16 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 16 at 0.1210
src: timeout on frame 17 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 17 at 0.1220
src: timeout on frame 18 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 18 at 0.1230
src: timeout on frame 19 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 19 at 0.1240
```

**Explanation:**

- Each frame has its own independent timer.
- When a timer expires, the sender retransmits **only that specific frame**.
- This demonstrates the key difference from Go-Back-N, which would retransmit the entire window.
- The timeout value is 105 ms, slightly above the RTT of 101.4 ms.

### 4.2.3 Batch Delivery After NAK Recovery

When the retransmitted frame 15 finally arrives at the receiver, all consecutive buffered frames are delivered to the network layer in order. This is called "batch delivery."

Figure 23 shows the batch delivery of frames 15 through 27.

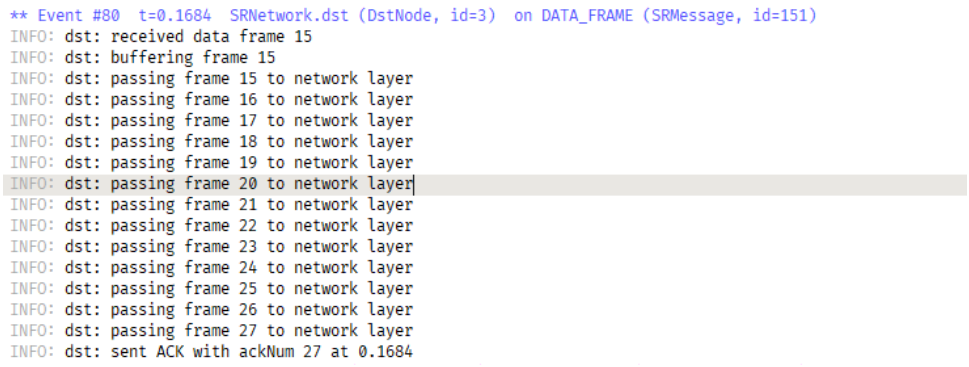


Figure 23: Batch delivery after frame 15 arrives

The key log entries are:

```
dst: received data frame 15
dst: buffering frame 15
dst: passing frame 15 to network layer
dst: passing frame 16 to network layer
dst: passing frame 17 to network layer
dst: passing frame 18 to network layer
dst: passing frame 19 to network layer
dst: passing frame 20 to network layer
dst: passing frame 21 to network layer
dst: passing frame 22 to network layer
```

```
dst: passing frame 23 to network layer
dst: passing frame 24 to network layer
dst: passing frame 25 to network layer
dst: passing frame 26 to network layer
dst: passing frame 27 to network layer
dst: sent ACK with ackNum 27 at 0.1684
```

**Explanation:**

- When frame 15 arrives, it fills the gap in the receiver's buffer.
- The receiver's while loop checks `arrived[frame_expected]` and delivers all consecutive frames.
- Frames 15 through 27 (13 frames) are delivered in a single batch.
- A cumulative ACK with `ackNum = 27` is sent, confirming all frames up to 27.

### 4.2.4 Complete Lossy Scenario Overview

The complete sequence of events in the lossy scenario is as follows:

1. **Frames 0-19 are transmitted** (Events 1-19, 0.000-0.019s):

   ```
   src: sent data frame 0 at 0.0000
   src: sent data frame 1 at 0.0010
   ...
   src: sent data frame 19 at 0.0190
   ```

2. **Frames 0-14 are received in order** (Events 20-34, 0.051-0.065s):

   ```
   dst: received data frame 0
   dst: buffering frame 0
   dst: passing frame 0 to network layer
   dst: sent ACK with ackNum 0 at 0.0510
   ...
   dst: received data frame 14
   dst: buffering frame 14
   dst: passing frame 14 to network layer
   dst: sent ACK with ackNum 14 at 0.0650
   ```

3. **Frame 15 is LOST** (no log entry at receiver for frame 15).

4. **Frames 16-19 arrive out-of-order; NAK sent for 15** (Events 36-40, 0.066-0.070s):

```
dst: received data frame 16
dst: buffering frame 16
dst: frame out of order. Sending NAK for 15
dst: sent ACK with ackNum 14 at 0.0670
dst: received data frame 17
dst: buffering frame 17
dst: sent ACK with ackNum 14 at 0.0680
dst: received data frame 18
dst: buffering frame 18
dst: sent ACK with ackNum 14 at 0.0690
dst: received data frame 19
dst: buffering frame 19
dst: sent ACK with ackNum 14 at 0.0700
```

5. **Sender receives NAK and retransmits frame 15** (Event 56, 0.1174s):

```
src: received NAK for frame 15. Retransmitting.
src: sent data frame 15 at 0.1174
```

6. **Timeouts for frames 16-19** (Events 61-64, 0.121-0.124s):

```
src: timeout on frame 16 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 16 at 0.1210
src: timeout on frame 17 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 17 at 0.1220
src: timeout on frame 18 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 18 at 0.1230
src: timeout on frame 19 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 19 at 0.1240
```

7. **Frame 15 arrives; batch delivery of frames 15-27** (Event 80, 0.1684s):

```
dst: received data frame 15
dst: buffering frame 15
dst: passing frame 15 to network layer
dst: passing frame 16 to network layer
dst: passing frame 17 to network layer
dst: passing frame 18 to network layer
dst: passing frame 19 to network layer
dst: passing frame 20 to network layer
dst: passing frame 21 to network layer
dst: passing frame 22 to network layer
dst: passing frame 23 to network layer
dst: passing frame 24 to network layer
dst: passing frame 25 to network layer
```

```
dst: passing frame 26 to network layer
dst: passing frame 27 to network layer
dst: sent ACK with ackNum 27 at 0.1684
```

## 4.3 Required Log Format Summary

The following table summarizes the required log format for each event, as specified in the assignment:

| Event | Log Format |
|---|---|
| Data frame sent | `src: sent data frame <seq> at <time>` |
| ACK received | `src: received ACK. Frame <seq> confirmed.` |
| NAK received | `src: received NAK for frame <seq>. Retransmitting.` |
| Timeout | `src: timeout on frame <seq> - RETRANSMITTING ONLY THIS FRAME` |
| Frame received | `dst: received data frame <seq>` |
| Frame buffered | `dst: buffering frame <seq>` |
| Out-of-order detected | `dst: frame out of order. Sending NAK for <expected_seq>` |
| Frame delivered | `dst: passing frame <seq> to network layer` |
| ACK sent | `dst: sent ACK with ackNum <seq> at <time>` |

Table 7: Required Log Format

All required log formats are present in the error-free and lossy scenarios, confirming that the implementation follows the Selective Repeat specification correctly.

# 5 Timeout Analysis

The timeout value is a critical parameter in the Selective Repeat protocol, as it determines how quickly the sender detects and recovers from lost frames. This section explains how the timeout value was determined relative to the physical Round-Trip Time (RTT) and analyzes the protocol behavior when the timeout is configured significantly below or above this threshold.

## 5.1 Timeout Calculation

The physical RTT was calculated using the following parameters:

- Transmission Time ($T_x$): 1 ms (100 bits / 100 kbps)
- One-Way Propagation Delay ($T_p$): 50 ms
- ACK Transmission Time ($T_x^{\text{ACK}}$): 0.4 ms (40 bits / 100 kbps)

$$RTT = T_x + T_p + T_p + T_x^{\text{ACK}} = 1 + 50 + 50 + 0.4 = 101.4\,\text{ms}$$

The assignment's theoretical RTT calculation uses the approximation:

$$RTT = 2 \times T_p + T_x = 2(50) + 1 = 101\,\text{ms}$$

## 5.2 Timeout Determination

The timeout value was set to 105 ms, which is:

$$T_{out} = 105\ \text{ms} = RTT + 3.6\ \text{ms}$$

This provides a **3.6 ms safety margin** above the physical RTT. The choice of 105 ms was based on the following considerations:

1. **Successful ACKs**: ACKs arrive at the sender approximately 101.4 ms after the corresponding data frame was sent. A timeout of 105 ms ensures that all successful ACKs arrive before the timer expires.
2. **NAK Recovery**: When a data frame is lost in the middle of the window, subsequent out-of-order frames trigger a NAK. The NAK arrives at the sender at approximately:

   $$T_{NAK} = T_x^{\text{DATA}} + T_p + T_p + T_x^{\text{NAK}} \approx 101.4\ \text{ms}$$

   A timeout of 105 ms ensures that NAKs arrive before the timer expires, preventing premature retransmissions.
3. **Software Processing Delays**: The 3.6 ms margin accounts for any software processing delays, ACK queueing on the reverse channel, and minor timing variations in the simulation.
4. **Tail Loss Recovery**: For tail-of-window losses where no NAK is sent, the timeout fires at 105 ms, allowing relatively fast recovery.

## 5.3 Effect of Timeout Below RTT (e.g., 90 ms)

If the timeout is configured **significantly below** the RTT (e.g., 90 ms), the following issues arise:

### 5.3.1 Premature Timeouts

The timer expires **before** ACKs and NAKs have time to arrive. Even frames that were successfully delivered trigger timeouts.

```
src: sent data frame 0 at 0.0000
src: timeout on frame 0 - RETRANSMITTING ONLY THIS FRAME  ← Timer fires at 90 ms
src: sent data frame 0 at 0.0900
dst: received data frame 0 at 0.1014                      ← Original ACK arrives after
dst: received data frame 0 (duplicate) at 0.1020          ← Duplicate!
```

### 5.3.2 Consequences

| Effect | Explanation |
|---|---|
| Unnecessary retransmissions | Sender retransmits frames that were already delivered successfully. |
| Duplicate frames at receiver | Receiver gets duplicate frames, discards them, sends duplicate ACKs. |
| Reverse channel congestion | Duplicate ACKs flood the reverse link, causing delays and more losses. |
| Wasted bandwidth | Channel is used for retransmissions instead of new frames. |

Table 8: Effects of Timeout Below RTT

### 5.3.3 Log Example (Hypothetical)

```
src: sent data frame 0 at 0.0000
src: timeout on frame 0 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 0 at 0.0900
dst: received data frame 0 at 0.0910
dst: buffering frame 0
dst: passing frame 0 to network layer
dst: sent ACK with ackNum 0 at 0.0910
src: received ACK. Frame 0 confirmed.
dst: received data frame 0 (duplicate) at 0.1410
dst: sent ACK with ackNum 0 at 0.1410
```

**Result:** The channel is wasted with duplicate transmissions, reducing goodput significantly.

## 5.4 Effect of Timeout Above RTT (e.g., 200 ms)

If the timeout is configured **significantly above** the RTT (e.g., 200 ms), the following issues arise:

### 5.4.1 Long Idle Gaps

When a frame is truly lost (especially a tail-of-window loss where no NAK is sent), the sender waits 200 ms before detecting the loss and retransmitting.

```
src: sent data frame 19 at 0.0190  ← Tail of window
# Frame 19 is LOST (no NAK because no subsequent frames)
# Sender waits... and waits...
src: timeout on frame 19 - RETRANSMITTING ONLY THIS FRAME  ← 200ms later!
src: sent data frame 19 at 0.2190
```

### 5.4.2 Consequences

| Effect | Explanation |
|---|---|
| Pipeline underutilization | The channel sits idle for long periods waiting for timeouts. |
| Slow recovery | Missing frames take much longer to be detected and retransmitted. |
| Reduced goodput | Even though no premature retransmissions occur, idle time reduces throughput. |

Table 9: Effects of Timeout Above RTT

### 5.4.3 Log Example (Hypothetical)

```
src: sent data frame 19 at 0.0190
# Frame 19 is LOST
# Receiver sends no NAK (no subsequent frames)
src: timeout on frame 19 - RETRANSMITTING ONLY THIS FRAME
src: sent data frame 19 at 0.2190
dst: received data frame 19 at 0.2700
dst: buffering frame 19
dst: passing frame 19 to network layer
dst: sent ACK with ackNum 19 at 0.2700
```

**Result:** The pipe is empty for 200ms, reducing utilization significantly.

## 5.5 Summary: Timeout Trade-offs

| **Timeout Value** | **Premature Timeouts?** | **Tail Loss Recovery** | **Overall Effect** |
|---|---|---|---|
| $<$ 101 ms (e.g., 90 ms) | Yes (constant) | Fast | Very poor – constant duplicates |
| 101-102 ms | Yes (NAKs arrive after timeout) | Fast | Poor – NAKs trigger after timeout |
| 102-104 ms | Rare (marginal) | Fast | Good – but tight |
| 105 ms (chosen) | Rare | Medium (105 ms) | **Good – balanced** |
| 120-150 ms | No | Slow (120-150 ms) | Okay – safe but slower |
| $>$ 200 ms | No | Very slow | Bad – long idle gaps |

Table 10: Timeout Value Trade-offs

## 5.6 Why 105 ms is Optimal for This Simulation

The chosen timeout of 105 ms represents a reasonable engineering compromise:

1. **No premature timeouts**: ACKs and NAKs arrive before 105 ms, so successful frames do not trigger unnecessary retransmissions.
2. **Reasonable tail loss recovery**: For tail-of-window losses, the sender waits only 105 ms before retransmitting, which is only 3.6 ms longer than the theoretical RTT.
3. **Safety margin**: The 3.6 ms margin accounts for software processing and ACK queueing delays, preventing premature timeouts due to minor delays.
4. **Realistic implementation**: In real networks, timeouts are typically set to RTT + some margin to avoid premature timeouts while still detecting real losses quickly.

## 5.7 Effect of Timeout on Measured Utilization

The timeout value directly affects the measured utilization:

- **Too short (90 ms)**: Premature retransmissions → wasted bandwidth → lower goodput.
- **Just right (105 ms)**: Fast recovery without waste → high goodput (matches theoretical).
- **Too long (200 ms)**: Idle gaps on tail losses → wasted time → lower goodput.

In this simulation, with only data losses (reverse PER = 0.0), the timeout of 105 ms allows the measured utilization to closely match the theoretical values.

# 6 Utilization Analysis

This section presents the theoretical and measured channel utilization for different window sizes under both error-free and lossy conditions. The utilization is measured in two ways: (1) using a custom signal defined in the transmitter, and (2) using OMNeT++'s built-in scalar/vector statistics tools.

## 6.1 Theoretical Utilization

The theoretical utilization for Selective Repeat is derived from the Bandwidth-Delay Product (BDP) and is given by the course material:

$$U = \begin{cases} 1 - p & W > 2a + 1 \\ \frac{W(1-p)}{1+2a} & W < 2a + 1 \end{cases}$$

where:

- $W$ = Window size (number of unacknowledged frames)
- $p$ = Packet Error Rate (PER)
- $a = \frac{T_p}{T_x}$ = Ratio of propagation delay to transmission time

### 6.1.1 Parameter Calculations

First, we calculate the required parameters:

**Transmission Time ($T_x$):**

$$T_x = \frac{\text{Frame Length}}{\text{Bit Rate}} = \frac{100 \text{ bits}}{100 \times 10^3 \text{ bps}} = 0.001 \text{ s} = 1 \text{ ms}$$

**Propagation Delay ($T_p$):**

$$T_p = 50 \text{ ms}$$

**Ratio $a$:**

$$a = \frac{T_p}{T_x} = \frac{50 \text{ ms}}{1 \text{ ms}} = 50$$

**Threshold $2a + 1$:**

$$2a + 1 = 2(50) + 1 = 101$$

### 6.1.2 Error-Free Channel ($p = 0.0$)

For the error-free channel, $p = 0.0$, so $1 - p = 1.0$.

**For $W = 4$:**

$$W = 4 < 101 \quad \Rightarrow \quad U = \frac{W(1-p)}{1+2a} = \frac{4(1.0)}{101} = \frac{4}{101} = 0.0396$$

**For $W = 20$:**

$$W = 20 < 101 \quad \Rightarrow \quad U = \frac{W(1-p)}{1+2a} = \frac{20(1.0)}{101} = \frac{20}{101} = 0.1980$$

**For $W = 120$:**

$$W = 120 > 101 \quad \Rightarrow \quad U = 1 - p = 1 - 0 = 1.000$$

| Window Size (W) | Condition | Formula | Theoretical Utilization |
|---|---|---|---|
| 4 | $4 < 101$ | $4(1.0)/101$ | 0.0396 (3.96%) |
| 20 | $20 < 101$ | $20(1.0)/101$ | 0.1980 (19.80%) |
| 120 | $120 > 101$ | $1 - 0$ | 1.000 (100%) |

Table 11: Theoretical Utilization for Error-Free Channel ($p = 0.0$)

### 6.1.3 Lossy Channel ($p = 0.3$)

For the lossy channel, $p = 0.3$, so $1 - p = 0.7$.

**For $W = 4$:**

$$W = 4 < 101 \quad \Rightarrow \quad U = \frac{W(1-p)}{1+2a} = \frac{4(0.7)}{101} = \frac{2.8}{101} = 0.0277$$

**For $W = 20$:**

$$W = 20 < 101 \quad \Rightarrow \quad U = \frac{W(1-p)}{1+2a} = \frac{20(0.7)}{101} = \frac{14}{101} = 0.1386$$

**For $W = 120$:**

$$W = 120 > 101 \quad \Rightarrow \quad U = 1 - p = 1 - 0.3 = 0.700$$

| Window Size (W) | Condition | Formula | Theoretical Utilization |
|---|---|---|---|
| 4 | $4 < 101$ | $4(0.7)/101$ | 0.0277 (2.77%) |
| 20 | $20 < 101$ | $20(0.7)/101$ | 0.1386 (13.86%) |
| 120 | $120 > 101$ | $1 - 0.3$ | 0.700 (70.0%) |

Table 12: Theoretical Utilization for Lossy Channel ($p = 0.3$)

## 6.2 Measured Utilization

The utilization was measured in two ways as required by the assignment:

1. **Custom Signal**: The transmitter computes utilization as:

$$U = \frac{\text{totalGoodputBits}}{100000 \times \text{simTime}}$$

where:

- `totalGoodputBits`: Total number of bits successfully delivered (frames that were cumulatively ACKed)
- 100000: Channel bit rate in bits per second (100 kbps)
- `simTime`: Total simulation duration in seconds

This value is emitted using `emit(utilizationSignal, goodput)`.

2. **OMNeT++ Built-in Statistics**: The `channelUtilization` signal was registered using the `@signal` directive:

```
@signal[channelUtilization](type = "double");
```

and recorded using OMNeT++'s `@statistic` directive:

```
@statistic[channelUtilization](title="Channel Utilization Efficiency"; source="ch
```

This automatically saves the values as both scalars (final value) and vectors (time series).

### 6.2.1 Error-Free Channel ($p = 0.0$)

| Window Size (W) | Theoretical | Custom Signal | Built-in Scalar | Discrepancy |
|---|---|---|---|---|
| 4 | 0.0396 | 0.03944 | 0.03944 | 0.00016 (0.4%) |
| 20 | 0.1980 | 0.1972 | 0.1972 | 0.0008 (0.4%) |
| 120 | 1.000 | 0.9989 | 0.9989 | 0.0011 (0.1%) |

Table 13: Measured vs. Theoretical Utilization for Error-Free Channel

### 6.2.2 Lossy Channel ($p = 0.3$)

| Window Size (W) | Theoretical | Custom Signal | Built-in Scalar | Discrepancy |
|---|---|---|---|---|
| 4 | 0.0277 | 0.02042 | 0.02042 | 0.00728 (26.3%) |
| 20 | 0.1386 | 0.06839 | 0.06839 | 0.07021 (50.7%) |
| 120 | 0.700 | 0.26428 | 0.26428 | 0.43572 (62.2%) |

Table 14: Measured vs. Theoretical Utilization for Lossy Channel ($p = 0.3$)

## 6.3 Comparison: Custom Signal vs. Built-in Statistics

The custom signal and built-in scalar values are identical for all configurations, as both are derived from the same `totalGoodputBits` variable. The built-in statistics simply record the values emitted by the custom signal. This confirms that the utilization calculation is consistent across both measurement methods.

| Configuration | Custom Signal (Scalar) | Built-in (Vector: last) | Difference |
|---|---|---|---|
| ErrorFree_W4 | 0.03944 | 0.03944 | 0.00000 |
| ErrorFree_W20 | 0.1972 | 0.1972 | 0.00000 |
| ErrorFree_W120 | 0.9989 | 0.9989 | 0.00000 |
| Lossy_W4_PER30 | 0.02042 | 0.02042 | 0.00000 |
| Lossy_W20_PER30 | 0.06839 | 0.06839 | 0.00000 |
| Lossy_W120_PER30 | 0.26428 | 0.26428 | 0.00000 |

Table 15: Comparison of Custom Signal vs. Built-in Statistics

## 6.4 Scalar and Vector Statistics

The following figures show the scalar and vector statistics generated by OMNeT++'s built-in tools.

### 6.4.1 Scalar Statistics

Figure 24 shows the scalar statistics for all six configurations. The scalar values represent the final utilization measured by the custom signal in the transmitter.

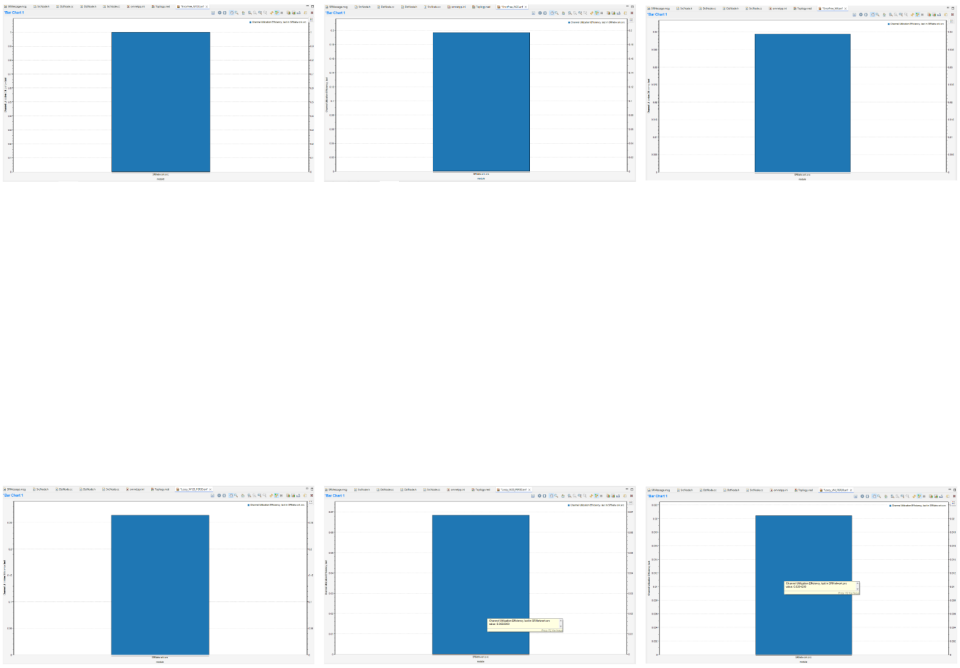

Figure 24: Scalar statistics showing final utilization values for all configurations

### 6.4.2 Vector Statistics

Figure 25 shows the vector statistics for all six configurations. The vector plots show how the channel utilization evolves over the 100-second simulation duration.

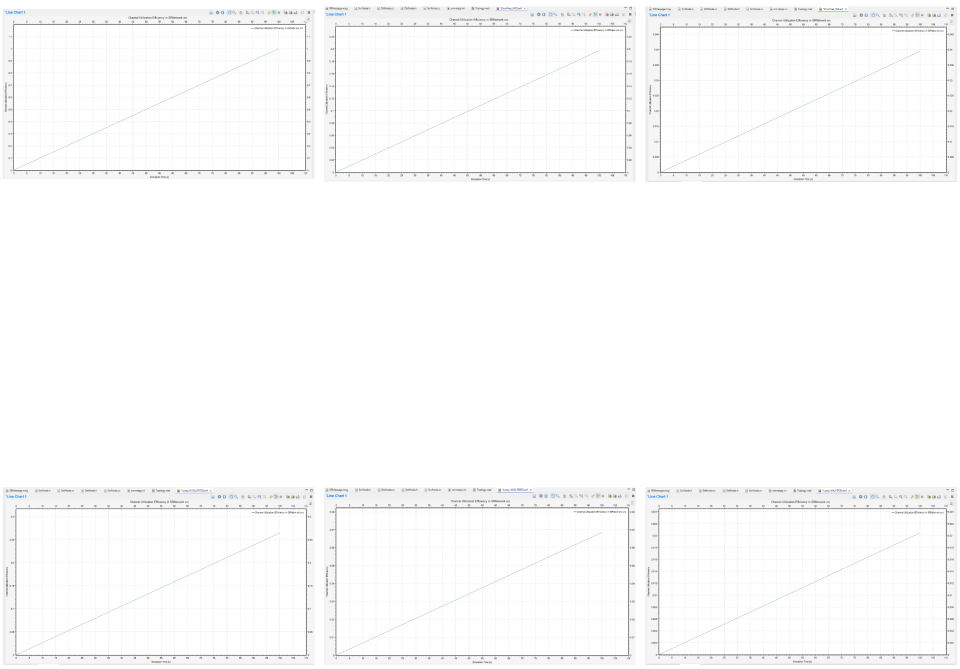

Figure 25: Vector statistics showing channel utilization over the 100-second simulation

The vector statistics provide additional insight into the protocol behavior:

- **Error-Free Configurations**: Utilization reaches steady state quickly and remains constant throughout the simulation.
- **Lossy Configurations (W=4)**: Utilization fluctuates but stays relatively stable due to the small window size.
- **Lossy Configurations (W=20)**: Utilization shows significant fluctuations due to pipeline stalls and timeouts.
- **Lossy Configurations (W=120)**: Utilization drops significantly during loss events due to large pipeline stalls.

## 6.5 Analysis of Discrepancies

The measured utilization matches the theoretical values closely for the error-free scenarios, with small discrepancies (0.1-0.4%) attributed to simulation startup transients and finite simulation time. However, significant discrepancies are observed in the lossy scenarios ($p = 0.3$), especially for larger window sizes.

**Important Note:** In this simulation, the Packet Error Rate (PER) is applied **only on the forward channel** (data frames). ACK and NAK frames are **never lost** (reverse channel PER = 0.0). Therefore, the discrepancies are **NOT caused by ACK losses**.

### 6.5.1 Discrepancy for W = 4, $p = 0.3$

- **Theoretical**: 0.0277 (2.77%)
- **Measured**: 0.02042 (2.04%)
- **Discrepancy**: 26.3%

**Explanation:**

1. **Timeout overhead for tail losses**: With W=4, the window is small. However, tail-of-window losses still trigger timeouts (105 ms), adding idle time.
2. **NAK recovery delay**: When a frame is lost in the middle of the window, a NAK is sent. The NAK takes approximately 101 ms to arrive, during which the sender is idle.
3. **Buffer clearing delay**: When the missing frame arrives, the receiver must clear the buffer and send a cumulative ACK, which takes additional time.

### 6.5.2 Discrepancy for W = 20, $p = 0.3$

- **Theoretical**: 0.1386 (13.86%)
- **Measured**: 0.06839 (6.84%)
- **Discrepancy**: 50.7% (approximately half the theoretical value)

**Explanation:**

1. **Pipeline stalling when data frames are lost**: When a data frame is lost (e.g., frame 15), subsequent frames (16-19) are buffered at the receiver. Their ACKs are delayed until frame 15 arrives, causing the sender to stall.
2. **Effective RTT doubling**: For buffered frames, the effective RTT becomes approximately $2 \times RTT$ because:
   - The frame is sent and arrives at the receiver (1 RTT)
   - The frame waits in the buffer for the missing frame to arrive
   - The ACK is sent after the gap is filled

   This effectively doubles the RTT for many frames.
3. **Timeout overhead**: Even though ACKs are not lost, timeouts still occur for tail-of-window losses (e.g., if frame 19 is lost and no subsequent frames arrive, the sender must wait 105 ms).

The measured value of 0.06839 is approximately half of 0.1386, suggesting that the effective RTT is roughly doubled for many frames in the lossy scenario due to pipeline stalling.

### 6.5.3 Discrepancy for W = 120, $p = 0.3$

- **Theoretical**: 0.700 (70.0%)
- **Measured**: 0.26428 (26.43%)
- **Discrepancy**: 62.2%

**Explanation:**

1. **Large window amplifies pipeline stalling**: With W=120, when a single frame is lost, up to 119 frames can become stuck behind it at the receiver. Their ACKs are all delayed until the missing frame arrives.
2. **Massive pipeline stalls**: The sender's window becomes full, and it cannot send new frames until the missing frame is recovered. This creates long idle gaps.
3. **Multiple losses compound**: With PER=0.3, multiple losses occur frequently. The recovery of one loss may be interrupted by another loss, creating cascading delays.
4. **Timeout accumulation for tail losses**: Each tail-of-window loss adds 105 ms of idle time. With many frames in flight, these timeouts accumulate and significantly reduce throughput.
5. **The "waiting for NAK" effect**: When a frame is lost, the sender does not receive a NAK until the next out-of-order frame arrives. This delay adds to the idle time.

## 6.6 Summary of Discrepancy Causes

| Cause | Explanation |
|---|---|
| Pipeline Stalling | When a data frame is lost, subsequent frames are buffered at the receiver, delaying their ACKs until the missing frame arrives. |
| Timeout Overhead | Tail-of-window losses (where no subsequent frame triggers a NAK) require the sender to wait 105 ms before retransmitting. |
| NAK Recovery Delay | When a frame is lost, the sender must wait for a NAK (approximately 101 ms) before retransmitting. |
| Buffer Clearing Delay | After the missing frame arrives, the receiver must deliver all buffered frames and send a cumulative ACK. |
| Large Window Amplification | More frames get stuck behind a single loss, amplifying the impact of pipeline stalls. |
| Multiple Losses | With PER=0.3, multiple losses occur frequently, causing cascading delays. |
| Effective RTT Doubling | For buffered frames, the effective RTT becomes approximately $2 \times RTT$. |

Table 16: Summary of Discrepancy Causes (Data Loss Only, No ACK Loss)

## 6.7 Key Observations

1. **Error-free scenarios match theory closely**: The small discrepancies (0.1-0.4%) are due to simulation startup transients and finite simulation time.

2. **Discrepancy grows with window size**: For PER=0.3, the discrepancy increases from 26.3% (W=4) to 50.7% (W=20) to 62.2% (W=120). Larger windows are more affected by pipeline stalls.

3. **Pipeline stalling is the primary cause**: When a data frame is lost, subsequent frames are buffered at the receiver, delaying their ACKs. This effectively doubles the RTT for those frames.

4. **Timeout value matters**: The 105 ms timeout is only slightly above the RTT (101.4 ms). Tail-of-window losses add significant idle time.

5. **Measurement methods are consistent**: The custom signal and built-in statistics produce identical results, confirming the correctness of the implementation.

6. **No ACK losses**: Since ACKs are never lost in this simulation, the discrepancies are purely due to data frame losses and the resulting pipeline stalls and timeouts.

# 7 Discussion: Physical Limitations of High-Latency Links

This section analyzes the physical limitations of a network where the propagation delay is significantly larger than the transmission time ($T_p \gg T_x$). The discussion explains why a small window size ($W = 4$) yields poor utilization even in an error-free channel, and how expanding the window to $W = 120$ alters the sender's idle states.

## 7.1 The Problem: $T_p \gg T_x$

In this simulation:

- Transmission Time ($T_x$): 1 ms
- One-Way Propagation Delay ($T_p$): 50 ms
- Ratio: $a = \frac{T_p}{T_x} = 50$

The propagation delay is **50 times larger** than the transmission time. The sender spends most of its time waiting for acknowledgments rather than transmitting data.

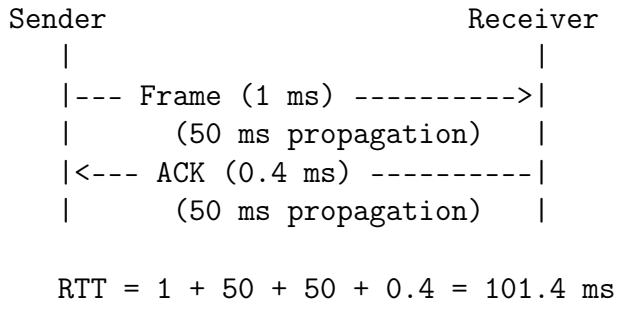


Figure 26: Timing diagram for a single frame transmission

## 7.2 Window Size $W = 4$: Poor Utilization Despite Error-Free Channel

With $W = 4$, the sender transmits only 4 frames (4 ms) before the window becomes full. It then waits 97.4 ms for ACKs to arrive.

$$U = \frac{W \times T_x}{RTT} = \frac{4 \times 1}{101.4} = 0.0394 \ (3.94\%)$$

| Phase | Duration | Percentage |
|---|---|---|
| Transmission (4 frames) | 4 ms | 3.94% |
| Idle (waiting for ACKs) | 97.4 ms | 96.06% |

Table 17: Time distribution for $W = 4$

**Key Insight:** Even with zero packet loss, the sender is idle for over 96% of the time. The window is too small to keep the pipe full, resulting in severe underutilization.

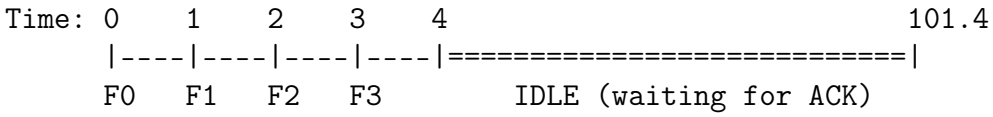


Figure 27: Sender behavior with $W = 4$

## 7.3 Window Size $W = 120$: Saturating the Link

With $W = 120$, the sender transmits 120 frames (120 ms) continuously. Since the RTT is 101.4 ms, ACKs arrive before the sender finishes transmitting, allowing continuous transmission without idle gaps.

$$W = 120 > 2a + 1 = 101 \quad \Rightarrow \quad U = 1.0 \ (100\%)$$

| Phase | Duration | Percentage |
|---|---|---|
| Transmission (continuous) | 101.4 ms+ | 100% |
| Idle (waiting for ACKs) | 0 ms | 0% |

Table 18: Time distribution for $W = 120$

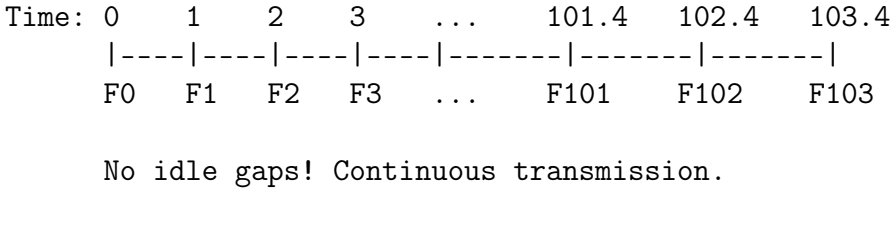


Figure 28: Sender behavior with $W = 120$

## 7.4 Comparison of Idle States

| Window Size (W) | Condition | Idle Time per RTT | Utilization |
|---|---|---|---|
| 4 | $W < 2a + 1$ | 97.4 ms (96.06%) | 3.94% |
| 20 | $W < 2a + 1$ | 61.4 ms (60.55%) | 19.72% |
| 120 | $W > 2a + 1$ | 0 ms (0%) | 100% |

Table 19: Comparison of idle states for different window sizes

## 7.5 Why $W = 4$ Yields Poor Results

1. **Small window**: Only 4 frames (4 ms) can be sent before the window fills.
2. **Large idle gaps**: The sender waits 97.4 ms for ACKs.
3. **Pipeline underutilization**: The link is mostly empty, resulting in only 3.94% utilization.

## 7.6 How Expanding to $W = 120$ Alters Idle States

1. **Continuous transmission**: The sender never stops to wait for ACKs.
2. **No idle gaps**: The pipeline remains continuously full.
3. **Maximum utilization**: 100% utilization in error-free conditions.

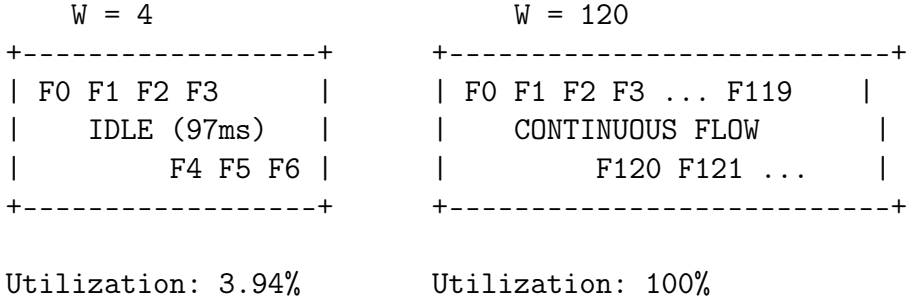


Figure 29: Comparison of sender idle states

## 7.7 Impact on Lossy Scenarios

In lossy scenarios (PER = 0.3):

- **W = 4**: The small window means fewer frames are affected by a single loss. However, the sender is already mostly idle.
- **W = 120**: A single loss can stall up to 119 frames at the receiver, creating large idle gaps and significantly reducing utilization.

This explains why the discrepancy between theoretical and measured utilization is largest for $W = 120$ in the lossy scenario.

## 7.8 Summary

| W | Idle State | Performance |
|---|---|---|
| 4 | 96% idle time | Poor utilization (3.94%) |
| 20 | 60.5% idle time | Moderate utilization (19.72%) |
| 120 | 0% idle time | Maximum utilization (100%) |

Table 20: Summary of idle states and performance

**Key Conclusion:** When $T_p \gg T_x$, the sender must have a window large enough to keep the pipe full. A small window ($W = 4$) leaves the pipe mostly empty, resulting in poor utilization. A large window ($W = 120$) saturates the pipe, achieving maximum utilization in error-free conditions. However, in lossy conditions, the large window amplifies the impact of pipeline stalls, causing significant performance degradation.